Coding and Implementation

Author: JahMari Harrigan. The implementation was developed with Codex assistance during this project session. The owner must review and explain the code. Development history records actual successive versions without fabricated dates, authorship or user feedback.

Version	Implemented behavior	Evidence
Alpha	Registration, database sessions, search/details, 1–4 passenger booking, lookup, cancellation, server-calculated fare and atomic allocations	alpha tag; evidence/alpha-tests.txt; 9 passing tests
Beta	Administrator catalog and scheduling; fare history protection; route analytics and CSV	beta tag; evidence/beta-tests.txt; 13 passing tests
Production classroom release	Synthetic analysis scenario; sign-in throttling/origin checks; full-capacity and schedule tests; clean setup and reports	Local production tag and history bundle; production-tests.txt and persistence-check.txt

Alpha behavior and limitations

The initial version used a normalized MySQL schema with customer operations and transaction protection. It ran through HTTP integration tests. Administrator functions and analytics were not yet present. The interface had not been visually verified in a browser. No real payment provider or email delivery was intended.

Beta improvements

First, administrators gained airport, aircraft, flight and dated-departure maintenance to satisfy operational requirements. Second, fare edits were tested against saved totals, preventing later changes from altering existing reservations. Third, protected route metrics and CSV export made the booking database useful for analysis. Tests increased from nine to thirteen; the schema remained stable.

Final Refinements and Verification

Refinement	Reason	Evidence
Sign-in throttling and origin validation	Reduce repeated authentication attempts and cross-origin writes	src/limits.js and server middleware; manual review
Reproducible synthetic analytics	Demonstrate SQL analysis with labeled provenance and explainable denominators	660 reservations; analytics-snapshot.json; analysis.sql
Capacity and schedule checks	Verify full flights, aircraft overlaps and cascading cancellation	T14 in production-tests.txt
Logout and relational constraints	Verify revoked sessions and MySQL FK/check rejection	T15–T16
Fresh-database setup	Avoid relying on hidden local state	clean-setup.txt: schema, seed and all 16 tests passed
Server restart persistence	Verify durable records and database sessions	persistence-check.txt: PASS T17 after Node.js restart

Observed errors and corrections

The first MySQL download URL returned 404, so the official archive package was used. pnpm encountered a sandbox path-resolution error; npm installed the locked dependencies successfully. A MySQL instance started in another isolated execution context was unreachable; running the server and tests together resolved connectivity. A later production test attempt failed for the same environment reason, then passed after the isolated preview was stopped. These were environment failures rather than successful tests and are recorded in the development log.

CLI refinement and remaining verification

On October 10, the owner completed a guided browser walkthrough; screenshots and reported checks are recorded in the evaluation report. A subsequent genuine Codex CLI session hid the lookup form only when printing. Five static checks passed; the owner then supplied a passing print preview. See evidence/cli-print-review.md. Earlier development used desktop Codex, not CLI. Three participant evaluations and the video remain incomplete; original version tags are preserved locally and in the history bundle.

Reproducible Setup and Code Explanation

Install Node.js and MySQL 8.4. Create a new database and project account, copy .env.example, supply local credentials and a demo password, then run npm ci, npm run setup and npm start. The README includes exact commands. The schema script does not erase an existing database; setup is intended for a fresh database. Synthetic demo accounts are customer@example.test and admin@example.test, with the locally configured DEMO_PASSWORD.

Explain these implementation decisions

Integer cents avoid floating-point fare rounding. A Flight defines a reusable route; a FlightInstance defines a dated departure. Booking stores the unit fare and total at reservation time. The database transaction locks inventory and commits all passengers together. A unique generated active-seat key prevents double allocation even if two requests compete. Cancellation preserves history while releasing active inventory. Session tokens are random, stored only as SHA-256 hashes, and linked to a database expiry; password hashes use salted scrypt.

Files to show during the video

Show schema.sql for keys and constraints, src/server.js for the reservation transaction and owner checks, src/admin.js for schedule maintenance and analytics, public/booking.html with app.js for the interface, tests/system.test.js for concurrent booking and rollback, and analytics/analysis.sql for CTEs and window functions. Use git log and the alpha/beta/production tags to explain the real version progression.

Production is the course release designation, not certification for commercial airline operations. No claims are made about real revenue, market performance, complete security auditing or accessibility compliance.